Programowanie współbieżne w C++

Lista zadań nr 1

Na pracownię 5 marca 2026

Zadanie 1 (5pkt). Problem producenta-konsumenta z ograniczonym buforem. Napisz program, który symuluje pracę wielu producentów oraz konsumentów korzystających z bufora o ograniczonej pojemności. Głównym zadaniem jest zapewnienie poprawnej synchronizacji wątków przy użyciu muteksów, tak aby nie występowały wyścigi ani zakleszczenia.

Szczegółowe wymagania:

1. Niech bufor będzie jednym ze standardowych kontenerów[1], np. `std::queue` lub `std::vector`.
2. Należy zadbać, by zawartość bufora nie przekroczyła ustalonej liczby elementów, np. 700.
3. Elementami bufora niech będą liczby całkowite losowane[2] przez producentów.
4. Uruchom np. po 13 wątków producentów i konsumentów. Konsument powinien wpisywać pobrany element do strumienia `std::cout`, który jest wątkowo bezpieczny (ale dopuszcza przeplot wartości wrzuconych weń w wielu wątkach).
5. Producenci i konsumenci w sytuacji napotkania odpowiednio pełnego i pustego bufora powinni zastosować mechanizm aktywnego oczekiwania (tzw. polling). Funkcje `produce` i `consume` powinny zwracać typ `bool` (sukces/porażka). W przypadku niemożności wykonania akcji, wątek powinien użyć `std::this_thread::yield()`, aby oddać czas procesora i spróbować ponownie. Wykorzystaj metodę `try_lock()` na muteksie, aby uniknąć zbędnego blokowania.
6. Użyj `std::mutex` (lub `std::recursive_mutex` w razie potrzeby)
7. Do zarządzania blokowaniem i odblokowywaniem muteksa stosuj idiom RAII (np. `std::lock_guard` lub `std::unique_lock`)[3].
8. Program powinien działać przez określony czas (np. 30 sekund) lub przetworzyć ustaloną liczbę elementów, po czym poprawnie zakończyć działanie wszystkich wątków.

---

[1] Kontenery nie są wątkowo bezpieczne, więc dostęp do nich należy synchronizować.
[2] Sprawdź zawartość nagłówka <random>
[3] W podobny sposób stosowaliśmy idiom RAII do automatycznego złączania wątków przy opuszczaniu zakresu przez obiekt typu std::thread. Zobacz też: C++ Concurrency in Action 2e, rozdział 3.

Zadanie 2 (5+1pkt). Symulacja przelewów między kontami bankowymi. Napisz program w C++ symulujący operacje na kontach bankowych, gdzie wiele wątków wykonuje jednocześnie losowe przelewy między kontami.

Szczegółowe wymagania:

1. Utwórz klasę `BankAccount` zawierającą:
   a. unikalny identyfikator konta (np. `int id`),
   b. aktualne saldo (np. `double balance`),
   c. muteks zabezpieczający dostęp do konta.
2. Zapewnij metody:
   a. `deposit(double amount)` – zwiększa saldo,
   b. `withdraw(double amount)` – zmniejsza saldo.
3. Zaimplementuj funkcję `transfer(BankAccount& from, BankAccount& to, double amount)`, która:
   a. sprawdza, czy na koncie źródłowym jest wystarczające saldo,
   b. wykonuje operację wypłaty z konta źródłowego oraz depozytu na koncie docelowym.
   c. oczekuje przez losową losową liczbę milisekund by zasymulować opóźnienie transferu (użyj `std::this_thread::sleep_for`)
4. Utwórz kilka (np. `700` lub więcej) kont bankowych z początkowym saldem, które razem dają stałą sumę.
5. Uruchom wiele wątków, z których każdy wykonuje w pętli losowe przelewy między losowo wybranymi kontami (np. losując kwotę i parę kont).
6. Po zakończeniu symulacji (np. po upływie określonego czasu lub wykonaniu ustalonej liczby operacji) sprawdź, że suma sald wszystkich kont jest równa sumie początkowej sprzed rozpoczęcia symulacji.
7. Komunikaty loguj na `std::cout`.

Wskazówka: Do zablokowania dwóch muteksów jednocześnie możesz użyć funkcji `std::lock()`. Zastanów się, dlaczego możesz chcieć to zrobić. Punkt bonusowy zdobędziesz, jeśli nie wykorzystasz ani tej ani żadnej innej funkcji bibliotecznej C++ umożliwiającej taką jednoczesną blokadę.

Zadanie 3 (5pkt). Zmodyfikuj poprzednie zadanie. Tym razem w symulacji mamy trzy typy wątków:

a) księgowi — wykonują losowe transfery tak, jak w poprzednim zadaniu
b) czytelnicy — jedynie odczytują stany losowych kont, jest ich znacznie więcej niż księgowych. Na tyle dużo, że jednocześnie wielu czytelników może chcieć przeczytać stan jednego konta

c) niecierpliwy audytor — pojedynczy wątek, który działa w cyklach. W każdym cyklu ma za zadanie zsumować pieniądze zgromadzone w banku[4]. Ponieważ audytor nie może pozwolić sobie na przestoje, daje on każdemu kontu tylko ograniczony czas (np. 10 milisekund) na uzyskanie dostępu do jego salda. Jeśli konto jest w tym czasie "zajęte", audytor go pomija i przechodzi do podliczania kolejnego. Do pominiętego konta będzie musiał wrócić w tym samym cyklu. (Zaktualizowane) Aby zapobiec głodzeniu (starvation), audytor po np. 5 nieudanych próbach powrotu do konta zgłasza błąd cyklu i przechodzi do kolejnego, chyba że wyczerpał ustalony z góry limit cykli.

Wskazówka: W tym zadaniu do ochrony kont bankowych powinieneś użyć innego narzędzia niż zwykły muteks, np. std::shared_timed_mutex (z C++14) wraz z odpowiednimi klasami opakowującymi: std::shared_lock do odczytu oraz std::unique_lock lub std::lock_guard do zapisu.

[4] Zauważ, że suma obliczona przez audytora działającego równolegle z księgowymi może nie być równa sumie początkowej.